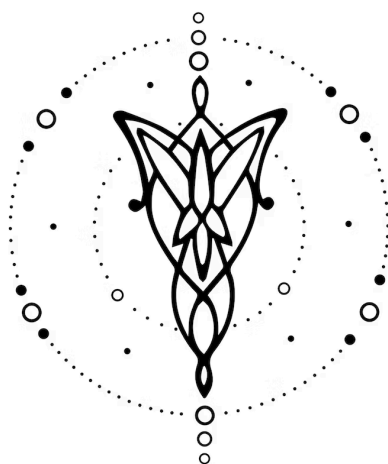




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Valentina	Fernández Flores	64142	vfernandezflores@itba.edu.ar
Julia	Priolo	63743	jupriolo@itba.edu.ar
Olivia	Rodrigué	64180	orodrigue@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [TPE-TLA](#)

3. Dominio

En la actualidad, la visualización de datos en entornos web suele implicar el manejo combinado de HTML, CSS y JavaScript, junto con la integración de librerías de terceros como Chart.js. Este proceso, aunque poderoso, resulta complejo y poco accesible para usuarios que no poseen conocimientos técnicos en desarrollo web, o que simplemente buscan una forma rápida y clara de representar datos sin invertir tiempo en aprender los detalles de dichas tecnologías.

En este contexto surge una necesidad arquitectural: disponer de un lenguaje específico de dominio (DSL) que permita describir de manera simple la creación de gráficos a partir de datos estructurados en formato CSV, para luego traducir automáticamente esa descripción a un código HTML válido y reutilizable. De esta forma, el usuario únicamente debe especificar el tipo de gráfico, los datos de entrada y algunas características básicas de estilo, sin preocuparse por la complejidad subyacente de HTML o JavaScript.

El dominio abstraído se centra en la simplificación de la creación de gráficos web para usuarios sin conocimientos de HTML/JS. Dicho DSL ofrece construcciones para generar de manera declarativa los principales tipos de gráficos utilizados en análisis y presentación de datos: gráficos de torta (pie chart), anillo (donut chart), barras (bar chart) y dispersión (scatter plot). Además, contempla opciones de estilo fundamentales como colores, títulos y leyendas, para que los resultados sean personalizables y útiles en contextos reales de comunicación.

Cabe señalar que este lenguaje no reemplaza a HTML o JavaScript de manera general, ya que su alcance está limitado a la generación de gráficos específicos. Inicialmente soporta solo los cuatro tipos de visualizaciones mencionados, aunque está diseñado de forma extensible para que, en un futuro, pueda incorporar nuevas categorías de gráficos y funcionalidades adicionales sin modificar la lógica de base.

La implementación satisfactoria de este lenguaje permitiría a usuarios no técnicos obtener visualizaciones claras y profesionales de sus datos en cuestión de segundos, minimizando errores manuales, ahorrando tiempo y facilitando la integración de gráficos en páginas web de forma reproducible y escalable.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán crear uno o varios gráficos dentro de un mismo programa.
- (II). Cada gráfico deberá especificar su tipo, pudiendo ser: pie, donut, bar o scatter.
- (III). Cada gráfico podrá asociarse a un archivo de datos en formato CSV, donde la primera fila representa etiquetas y las filas subsiguientes representan valores.
- (IV). Se podrá asignar un título a cada gráfico.
- (V). Se podrá personalizar la paleta de colores utilizada en los gráficos.

- (VI). Se podrá activar o desactivar la leyenda, así como modificar su posición (por ejemplo: arriba, abajo, izquierda, derecha).
- (VII). Se podrán declarar múltiples gráficos en un mismo programa, los cuales serán renderizados en conjunto dentro del archivo HTML generado.
- (VIII). Se podrá especificar un identificador único para cada gráfico, de forma que pueda ser referenciado o reutilizado dentro del HTML generado.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

1. Un programa que construya un gráfico de barras a partir de un archivo `ventas.csv` con etiquetas y valores.
2. Un programa que construya un gráfico de torta (pie chart) mostrando la proporción de 3 categorías.
3. Un programa que construya un gráfico de dispersión (scatter plot) con un conjunto de pares ordenados de datos.
4. Un programa que construya un gráfico de anillo (donut chart) con etiquetas y valores, aplicando colores personalizados.
5. Un programa que construya dos gráficos en un mismo programa (por ejemplo, un pie chart y un bar chart) y los genere en un mismo HTML.
6. Un programa que construya un gráfico de barras con un título definido por el usuario.
7. Un programa que construya un gráfico de torta con una leyenda activada y ubicada en la parte inferior.
8. Un programa que construya un gráfico de dispersión, donde el usuario debe especificar un identificador único que permitirá su posterior inserción dentro de otra página web.
9. Un programa que genere correctamente un HTML válido con todos los elementos necesarios para visualizar el gráfico sin intervención adicional del usuario.
10. Un programa que, dado un archivo CSV válido, genere un gráfico reproducible en múltiples ejecuciones.
11. Un programa que, dadas 3 variables elegidas genere un gráfico de barras en el cual el usuario especifique un color de la paleta y el sistema complete automáticamente los colores faltantes de manera aleatoria.

Además, los siguientes casos de prueba de **rechazo**:

1. Un programa con errores de sintaxis en el DSL.
2. Un programa que intente construir un gráfico sin especificar el tipo.
3. Un programa que intente utilizar un archivo CSV inexistente.
4. Un programa que intente generar un gráfico con un tipo no soportado (por ejemplo, line o radar).
5. Un programa que declare un mismo identificador de gráfico más de una vez.
6. Un programa que declare variables que no existan en el archivo CSV

6. Ejemplos

Creación de gráfico de barras con identificador `grafico_ventas`, que relaciona mes y monto, con título “Ventas 2024”, a partir del archivo “ventas.csv”.

```
chart "Ventas 2024" type bar:
  from "./ventas.csv",
  x = "Mes",
  y = "Monto",
  orientation = vertical,
  colors = ["#4CAF50", "#2196F3"],
  id = "grafico_ventas";
```

Creación de gráfico de dispersión que relaciona *peso* y *altura*, definiendo los rangos de las mismas, a partir del archivo “datos_personas.csv”, el identificador es `grafico_altura_peso`.

```
chart "Altura vs Peso" type scatter:
  from "datos_personas.csv",
  x = "Peso",
  y = "Altura",
  x.range = [50, 100],
  y.range = [150, 200],
  color = red,
  id = "grafico_altura_peso";
```

Creación de gráfico de torta que relaciona tipos de tartas y cantidad a partir del archivo `tartas_2025.csv`. La leyenda serían las etiquetas de cada porción del gráfico, y en este caso se ubicaría en la parte de abajo del gráfico.

```
chart "Tartas 2025" type pie:
  from "tartas_2025",
  x = "Tipo de tartas",
  y = "Cantidad",
  legend = bottom,
  id = "tipo_tartas";
```

Creación de gráfico de anillo titulado “Distribución de gastos mensuales” a partir del archivo `categorias.csv`. El parámetro `hole = 0,2` indica que el agujero central ocupará el 20 % del radio total del gráfico.

```
chart "Distribución de gastos mensuales" type donut
  from "gastomensuales.csv"
```

```
x = "Gastos"
y = "Porcentaje"
hole = 0,2
id = "grafico_donut";
```